Alexandria University
Specialized Scientific Programs
Computer and Communication Department



Natural Language Processing
Instructor: Prof. Dr. Ayman Khalafallah
Deadline: Tuesday 20/10/2026 at 11:59 PM

Assignment 1: From N-Grams to Logistic Regression

1. Assignment Overview

In this assignment, you will build a complete Natural Language Processing (NLP) pipeline **entirely from scratch using Python and NumPy**. You will explore how foundational algorithms: Language Models, Naive Bayes, and Logistic Regression, build upon one another to solve a real-world text classification problem.

Learning Objectives:

- Implement text pre-processing, tokenization, and N-Gram extraction without external libraries.
- Build class-conditional Bigram Language Models and compute Log-Likelihood and Perplexity.
- Build a Multinomial Naive Bayes classifier from scratch using Add-1 (Laplace) smoothing.
- Understand and calculate Naive Bayes Log-Ratios.
- Train a Logistic Regression classifier from scratch using Gradient Descent.
- Analyze the theoretical differences between Generative and Discriminative models.

Rules & Constraints:

- **No Machine Learning Libraries:** You may **not** use `scikit-learn`, `nltk`, `spacy`, `tensorflow`, or `pytorch` for any of the text cleaning, feature extraction, or modeling. You must use standard Python and `numpy`.
- You are permitted to use the Hugging Face `datasets` library solely to download the raw dataset.

2. Dataset Setup

We will use the **IMDB Movie Reviews Dataset**, a classic binary classification dataset.

To keep training times fast while using pure NumPy, you should not use the entire dataset. You are required to extract a balanced subset consisting of exactly **5,000 training reviews** (2,500 positive, 2,500 negative) and **1,000 test reviews** (500 positive, 500 negative).

3. Assignment Tasks

Part 1: Pre-processing, Language Models, and Perplexity

In this part, you will build and evaluate true language models for each sentiment class.

1. **Data Pre-processing & Tokenization:** Inspect the raw dataset and determine the necessary text cleaning steps required. Implement your chosen pre-processing pipeline, then write a tokenizer to split the cleaned text into discrete words. Make sure to pad your sequences with start and end tokens (e.g., `<s>` and `</s>`).
2. **Class-Conditional Language Models:** Divide your training data by class. Extract Bigrams and build two separate language models: a **Positive Bigram LM** and a **Negative Bigram LM**. Use Add-1 (Laplace) smoothing to handle unseen bigrams.
3. **Log-Likelihood & Perplexity:** Write a function that takes a new document and calculates its **Log-Likelihood** and **Perplexity** under both the Positive and Negative LMs.
4. **LM Prediction:** Use your language models as a classifier. For every review in the test set, predict the class based on which model yields the *lower perplexity*. Calculate the Accuracy and F1-Score of this Generative LM approach.
5. **LM Inference:** Prepare a function that applies "Next Word Prediction" given a sentence untill it reaches </s> token or reaches some given maximum length.

Part 2: Vocabulary, Naive Bayes, and Log-Ratios

Instead of relying on the sequence structure, you will now treat documents as bags-of-N-Grams to compute independent sentiment scores.

1. **Build a Vocabulary:** Iterate through your training data and find the 2,000 most frequent Unigrams and Bigrams combined. Create a dictionary mapping these 2,000 features to unique integer indices. Ignore all other words outside this vocabulary.
2. **Calculate Probabilities:** Count the occurrences of each vocabulary feature in the Positive and Negative classes. Apply Laplace (Add-1) smoothing to calculate $P(\text{feature} \mid \text{Positive})$ and $P(\text{feature} \mid \text{Negative})$.
3. **Compute Log-Ratios (r):** For each of your 2,000 features, calculate its Naive Bayes Log-Ratio:
$$r_j = \log\left(\frac{P(\text{feature}_j \mid \text{Positive})}{P(\text{feature}_j \mid \text{Negative})}\right)$$
4. **Naive Bayes Prediction:** Write a function that predicts the class of a document by summing the log-ratios of the vocabulary words it contains (do not forget the class prior ratio). Calculate your baseline Accuracy and F1-Score on the test set.

Part 3: NB-Weighted Logistic Regression

Generative models (like Naive Bayes) suffer from the "conditional independence assumption." By using both Unigrams and Bigrams, we heavily violate this assumption. We will use Logistic Regression to learn the optimal combination of features.

1. **Feature Matrix Construction:** Build a feature matrix X of shape `(num_documents, 2000)`. Multiply the count of the feature in the document by its Log-Ratio r_j to provide the model with dense sentiment signals:
$$X_{i,j} = \text{count}(\text{feature}_j \text{ in } \text{doc}_i) \times r_j$$
2. **Standardization:** Standardize your feature matrix (Z-score normalization) and add a Bias column of 1s.
3. **Gradient Descent:** Implement a Logistic Regression class from scratch. Write a training loop using Gradient Descent to minimize Binary Cross-Entropy Loss. Train this model on your standardized X matrix.

Part 4: Evaluation and Analysis

Model Comparison: In a markdown cell, compare the three approaches used in the assignment: **Bigram Language Model**, **Naive Bayes**, and **Logistic Regression**. Include comparisons of the input representation, accuracy and F1 score.

Briefly discuss which model performed best and provide a possible explanation for the observed differences in performance.

4. Final Deliverables

You must submit exactly **ONE file**: a Google Colab Notebook (`.ipynb` file).

Your notebook must include:

1. All executable code logically organized into code cells.
2. Printed outputs (do not clear your cell outputs before submitting) showing your training progress, loss decreasing during gradient descent, and final accuracy/F1-score tables.
3. A Markdown cell at the very end containing your written analysis for Part 4.